Supplemental Materials

Toward Model-Free Dexterous Manipulation with Low-Cost Tactile Sensors: Learning to Control Normal Contact Force

Jinda Cui† and Jeff Trinkle†

A. Analysis of the Chiral Problem

In the paper, we introduced the chiral problem, which flips the sign of $\dot{f}_n$ at different contact regions. Without further analysis, we could throw this problem to machine learning. However, it is crucial to understand if the contact region information can be inferred at all. Otherwise, the learning may be destined to failure. Here, we take a closer look into this problem.

What the learned policy regulates is essentially the relative normal speed between the two contacting surfaces. Let us denote the relative normal speed as v_{rel} and the normal velocity offset at the contact as v_{ofs} (note that this is a linear velocity offset resulted from a_{ofs}).

Fig. 1 enumerates all possible combinations of contact regions and relative velocity directions. If the robot knows $|v_{rel}|$, it can always generate a v_{ofs} bigger than $|v_{rel}|$ and check the sign of $\dot{f}_n$. In this case, if v_{ofs} and $\dot{f}_n$ have the same sign, the contact is in one region; if the sign is different, the contact is for sure in the other region. However, v_{rel} is not a measured value (and is difficult to measure). The policy can only observe its immediate previous action and the resulting force change, $\dot{f}_n$, not v_{rel} or $|v_{rel}|$.

Now depending on the relationship between v_{ofs} and $|v_{rel}|$, we can divide the scenarios into three segments for each of the four contact cases. The segments are listed below each diagram. In each table, the first row states the three-scenario segments. The second row reflects the sign of $\dot{f}_n$ resulting from the action. For each segment, the sign of $\dot{f}_n$ is deterministic, and we may use the sign of v_{ofs} and $\dot{f}_n$ as a feature to infer the contact region. The third row is the extracted feature, $sign(\dot{f}_n v_{ofs})$. For the first and third scenario segments, the feature states the contact is in the left region if it is -1 and the right region if 1. This conflicts with the second scenario segment, which states the opposite. Thus, the controller cannot distinguish the contact regions with the inputs used in the examples.

In our implementation, v_{ofs} is not observable. However, the above conclusion still holds if we use a_{ofs} to replace v_{ofs} in the feature, since $sign(v_{ofs}) = sign(a_{ofs})$.

Following the same analysis, we noticed that when the $\ddot{f}_n$ is also given, the contact region can be determined. However, this observation is noisy in simulation, probably due to integration errors, and especially noisy in the hardware robot system when the communication latency and noise come into play. Instead of relying on the $\ddot{f}_n$, an alternative is to let the policy observe a sequence of previous action-response pairs. This would also be useful if the policy learned "probing" actions that may embed information in the sequence.

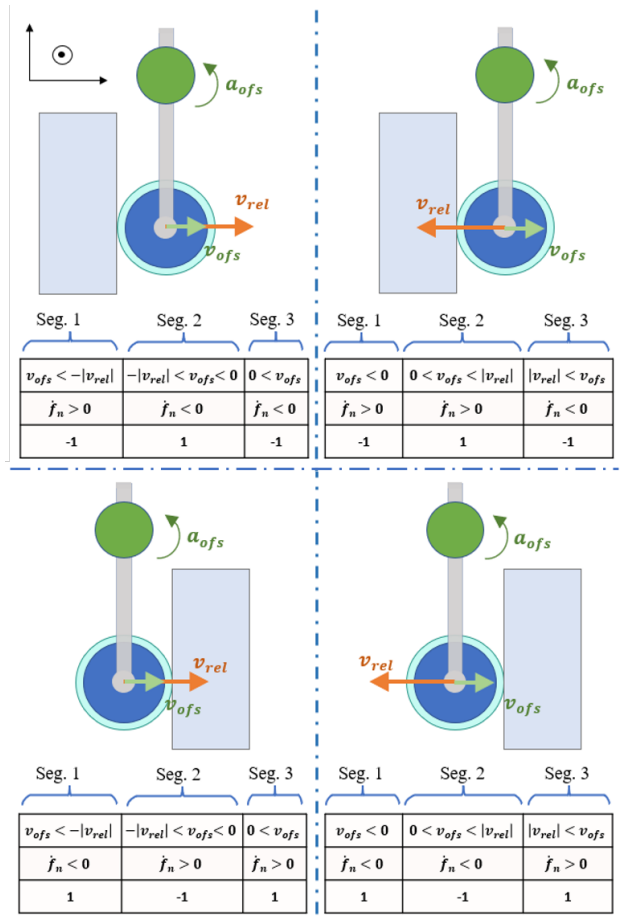

Fig. 1. All scenarios of contact regions and relative velocity directions.

B. Benchmark 1

1) The choice of T: The variable T in the mean absolute off-set determines how many timesteps we measure after the initial change of the target force, and the choice of T

†The authors are with the Department of Computer Science and Engineering, Lehigh University, Bethleham, PA, USA.

affects the reported controller performance. Let us look at two typical control plots of the controllers in comparison in Fig. 2.

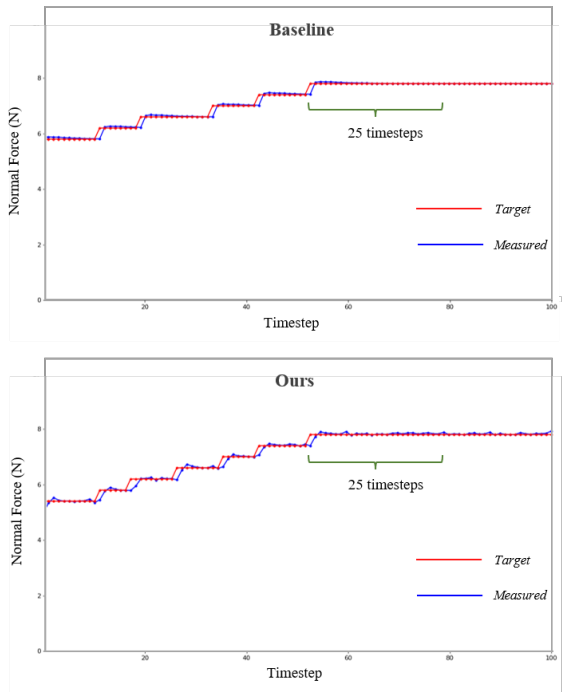

Fig. 2. Typical control measurements during force increments for the baseline and our learned policy.

When the step-change in target force happens, there can be a delay in response from the controllers, and the rise time following that is non-negligible. We want fast-responding controllers, but speed may come with a cost - overshooting. From our observations, the initial transients before reaching steady-state are often within 5-10 timesteps for either controller. Increasing T will start to blend in the steady-state performances, and further increment of T will bury the initial transient information.

The baseline has near-zero steady-state errors. By choosing $T = 25$, we do a pessimistic evaluation of our learned policy and give the upper hand slightly to the baseline, while retaining some information about the transients.

2) Real-world Evaluations: The reality gap presents between the simulation and the hardware robotic system. On the hardware, we observed communication latency, measurement noise, and actuation delay. The control frequency is set conservatively to 5Hz as a consequence. In addition, we have to impose a joint speed limit to prevent the system from damaging itself from wild actions. We plan to perform in-depth studies regarding the reality gap in the future.

Despite the imperfections on the hardware system, the learned policy can still demonstrate its abilities in the real world, to an extent. We performed the first benchmark in the real world. The tactile sensor is pressed against a heavy subwoofer, designed to be vibration-resistant and very stiff. The contact region is padded with a thin foam tape (same as the black stripe on the rectangular block in Fig. 1 of the paper). Moreover, the contact angle is guided by a precisely machined angle finder.

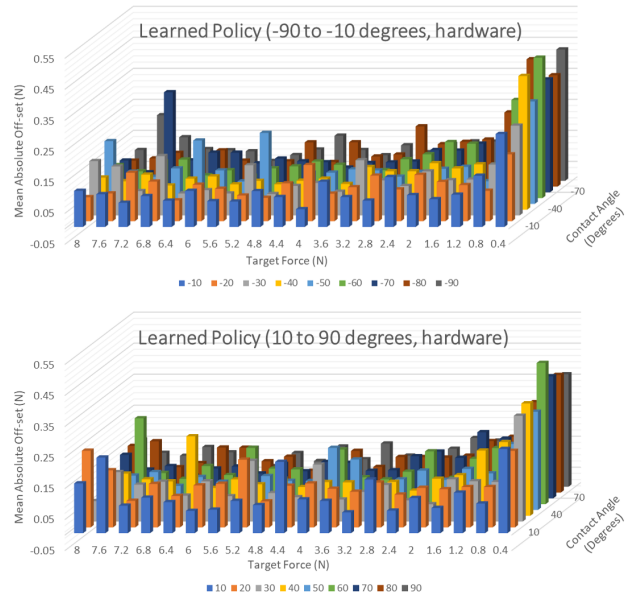

Fig. 3. Benchmark 1, results with the hardware robotic system.

The results are summarized in Fig. 3. The overall average score is 0.133, which is 1.66 times the score obtained in the simulation. Similar to the simulation results, the controller performed similarly across the angle variations and the force range. Moreover, it also suffers from performance degradation at a very low target force. However, in our experience, the force controller is still usable at low contact angles with low target forces in manipulation tasks.

C. Benchmark 2 - Real-World Results

As mentioned in the paper, evaluating the disturbance tolerance on the hardware system is risky, as losing control over the contact force may break the robot. In fact, we broke the end-effector apart in one of the tests. For this reason, instead of letting the disturbances press into the contact, we use disturbances that separates the contact. However, this measurement can be conservative because a large disturbance may break the contact before the controller could respond.

In the experiments, we use the pseudo-inverse of the robot Jacobian to compute the joint velocities corresponding to the separating normal velocity, then let the robot execute it. The benchmark results are reported in Fig. 4. We did not observe any of the maximum disturbances to break the contact before the controller kicked in. The disturbance tolerance on the hardware system is comparable to the simulation results, but the performance drops are visible.

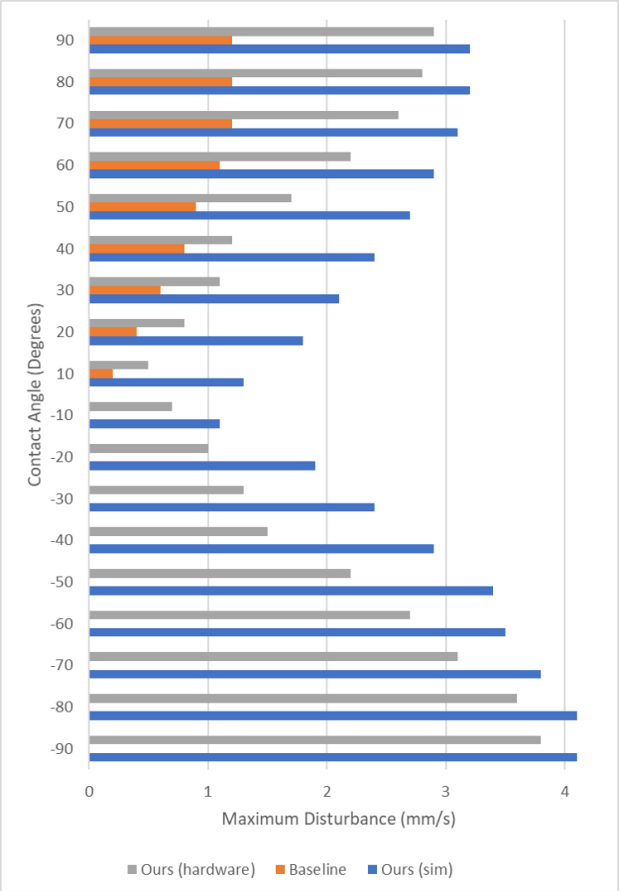

Fig. 4. Benchmark 2, results from the hardware robotic system compared to previous results.